

1. Limbajul Prolog

- Limbajul Prolog (PROgrammation en LOGique) a fost elaborat la Universitatea din Marsilia în jurul anului 1970, ca instrument pentru programarea și rezolvarea problemelor ce implicau reprezentări simbolice de obiecte și relații dintre acestea.
- Prolog are un câmp de aplicații foarte larg: baze de date relaționale, inteligență artificială, logică matematică, demonstrarea de teoreme, sisteme expert, rezolvarea de probleme abstracte sau ecuații simbolice, etc.
- Există standardul ISO-Prolog.
- Nu există standard pentru programare orientată obiect în Prolog, există doar extensii: TrincProlog, SWI-Prolog.
- Vom studia implementarea SWI-Prolog – sintaxa e foarte apropiată de cea a standardului ISO-Prolog.
 - Turbo Prolog, Visual Prolog, GNUProlog, Sicstus Prolog, Parlog, etc.
- SWI-Prolog – 1986
 - oferă o interfață bidirecțională cu limbajele C și Java
 - folosește XPCE – un sistem GUI orientat obiect
 - *multithreading* – bazat pe suportul *multithreading* oferit de limbajul standard C.

Program Prolog

- caracter descriptiv: un program Prolog este o colecție de definiții ce descriu relații sau funcții de calculat – reprezentări simbolice de obiecte și relații între obiecte. Soluția problemelor nu se mai vede ca o execuție pas cu pas a unei secvențe de instrucțiuni.
 - program – colecție de declarații logice, fiecare fiind o clauză Horn de forma $p, p \rightarrow q, p_1 \wedge p_2 \dots \wedge p_n \rightarrow q$
 - **concluzie** de demonstrat – de forma $p_1 \wedge p_2 \dots \wedge p_n$
- **Structura de control** folosită de interpretorul Prolog
 - Se bazează pe declarații logice numite **clauze**
 - **fapt** – ceea ce se cunoaște a fi adevărat
 - **regulă** - ce se poate deduce din fapte date (indică o concluzie care se știe că e adevărată atunci când alte concluzii sau fapte sunt adevărate)
 - **Concluzie** ce trebuie demonstrată - GOAL
 - Prolog folosește *rezoluția* (liniară) pentru a demonstra dacă concluzia (teorema) este adevărată sau nu, pornind de la ipoteza stabilită de faptele și regulile definite (axiome).
 - Se aplică raționamentul înapoi pentru a demonstra concluzia
 - Programul este citit de sus în jos, de la dreapta la stânga, căutarea este în adâncime (*depth-first*) și se realizează folosind **backtracking**.

- $p \rightarrow q$ se transcrie în Prolog folosind clauza $q :- p.$ (q if $p.$)
- $p_1 \wedge p_2 \dots \wedge p_n \rightarrow q$ se transcrie în Prolog folosind clauza $q :- p_1, p_2, \dots, p_n.$
 - $\wedge$ se transcrie în Prolog folosind ","
 - $\vee$ se transcrie în Prolog folosind ";" sau o clauză separată.

Exemple

• Logică

$\forall x p(x) \wedge q(x) \rightarrow r(x)$

$\forall x w(x) \vee s(x) \rightarrow p(x)$

$\forall x t(x) \rightarrow s(x) \wedge q(x)$

$t(a)$

$w(b)$

Concluzie $r(a)$

Prolog

$r(X) :- p(X), q(X).$

$p(X) :- w(X).$

$p(X) :- s(X).$

$s(X) :- t(X).$

$q(X) :- t(X).$

$t(a).$

$w(b).$

• Logică

$\forall x s(x) \rightarrow p(x) \vee q(x)$

Prolog

????

2. Elemente de bază ale limbajului SWI-Prolog

1. Termen

▪ SIMPLU

a. constantă

- simbol (*symbol*)
 - secvență de litere, cifre, _
 - începe cu **literă mică**
- număr =întreg, real (*number*)
- șir de caractere (*string*): 'text' (caracter: 'c', '\t',...)

ATOM = SIMBOL + STRING +ȘIR-DE-CARACTERE-SPECIALE (<--->,===>) + []

b. variabilă

- secvență de litere, cifre, _
- începe cu **literă mare**
- variabila anonimă este reprezentată de caracterul underline (_).

▪ COMPUS (a se vedea Secțiunea 11).

- listele (*list*) sunt o clasă specială de termeni compuși

2. Comentariu

```
% Acesta este un comentariu
/* Acesta este un comentariu */
```

3. Predicat

- a). standard (ex: **fail**, **number**, ...)
- b). utilizator

- $\text{nume}[(\text{obiect}[, \text{obiect}....])]$
↓
numele simbolic al relației

Tipuri

1. **number** (integer, real)
2. **atom** (symbol, string, șir-de-caractere-speciale)
3. **list** (secvență de elemente) specificat ca **list**=tip_de_bază*

ex. listă (omogenă) formată din numere întregi [1,2,3]

% definire tip: el=integer list=el*

!!! lista vidă [] este singura listă care e considerată în Prolog atom.

Convenții.

- În SWI-Prolog nu există declarații de predicate, nici declarații de domenii (definiții de tipuri - ca în Turbo-Prolog).
- *Specificarea unui predicat*
 - % definire tipuri, dacă e cazul
 - % *nume* [(param₁:tip₁[,param₂:tip₂...)]
 - % modelul de flux al predicatului (i, o, ...) - vezi Secțiunea 4
 - % param₁ - semnificația parametrului 1
 - % param₂ - semnificația parametrului 2
 -

4. Clauza

- fapt
 - relație între obiecte
 - *nume_predicat* [(obiect [, obiect....)]
- regula
 - permite deducere de fapte din alte fapte

Exemplu:

fie predicatele

tata(X, Y) reprezentând relația “Y este tatăl lui X”
mama(X, Y) reprezentând relația “Y este mama lui X”

și următoarele fapte corespunzătoare celor două predicate:

mama(a,b).
mama(e,b).
tata(c,d).
tata(a,d).

Se cere: folosind definițiile anterioare să se definească predicatele

parinte(X, Y) reprezentând relația “Y este părintele lui X”
frate(X, Y) reprezentând relația “Y este fratele lui X”

Clauze în Prolog

parinte(X,Y) :-tata(X,Y).
parinte(X,Y) :-mama(X,Y).
frate(X,Y) :-parinte(X,Z),parinte(Y,Z),X\=Y.

5. Întrebare (goal)

○ e de forma $predicat_1 [(obiect [, obiect....)], predicat_2 [(obiect [, obiect....)]....]$.

Folosind definițiile anterioare, formulăm următoarele întrebări:

?- parinte(a,b).	?- parinte(a,X).
true.	X=d;
	X=b.

? - parinte(a,f).
false.

?- frate(a,X).	?- frate(a,_).
X=c;	true.
X=e.	

3. “Matching”. Cum își primesc valori variabilele?

Prolog nu are instrucțiuni de atribuire. Variabilele în Prolog își primesc valorile prin **potrivire** cu constante din fapte sau reguli.

Până când o variabilă primește o valoare, ea este **liberă** (free); când variabila primește o valoare, ea este **legată** (bound). Dar ea stă legată atâta timp cât este necesar pentru a obține o soluție a problemei. Apoi, Prolog o dezleagă, face backtracking și caută soluții alternative.

Observație. Este important de reținut că nu se pot stoca informații prin atribuire de valori unor variabile. Variabilele sunt folosite ca parte a unui proces de potrivire, nu ca un tip de stocare de informații.

Ce este o potrivire?

Iată câteva reguli care vor explica termenul 'potrivire':

1. Structuri identice se potrivesc una cu alta
 - $p(a, b)$ se potrivește cu $p(a, b)$
2. De obicei o potrivire implică variabile libere. Dacă X e liberă,
 - $p(a, X)$ se potrivește cu $p(a, b)$
 - X este legat la b .
3. Dacă X este legat, se comportă ca o constantă. Cu X legat la b ,
 - $p(a, X)$ se potrivește cu $p(a, b)$
 - $p(a, X)$ NU se potrivește cu $p(a, c)$
4. Două variabile libere se potrivesc una cu alta.
 - $p(a, X)$ se potrivește cu $p(a, Y)$

Observație. Mecanismul prin care Prolog încearcă să 'potrivească' partea din întrebare pe care dorește să o rezolve cu un anumit predicat se numește **unificare**.

4. Modele de flux

În Prolog, legările de variabile se fac în două moduri: la intrarea în clauză sau la ieșirea din clauză. Direcția în care se leagă o valoare se numește '**model de flux**'. Când o variabilă este dată la intrarea într-o clauză, aceasta este un parametru de intrare (i), iar când o variabilă este dată la ieșirea dintr-o clauză, aceasta este un parametru de ieșire (o). O anumită clauză poate să aibă mai multe modele de flux. De exemplu clauza

`factorial (N, F)`

poate avea următoarele modele de flux:

- (i,i) - verifică dacă $N! = F$;
- (i,o) - atribuie $F := N!$;
- (o,i) - găsește acel N pentru care $N! = F$.

Observație. Proprietatea unui predicat de a funcționa cu mai multe modele de flux depinde de abilitatea programatorului de a programa predicatul în mod corespunzător.

5. Sintaxa regulilor

Regulile sunt folosite în Prolog când un fapt depinde de succesul (veridicitatea) altor fapte sau succesiuni de fapte. O regulă Prolog are trei părți: capul, corpul și simbolul `if (:-)` care le separă pe primele două.

Iată sintaxa generică a unei reguli Turbo Prolog:

capul regulii :-

subgoal,
subgoal,
...,
subgoal.

Fiecare subgoal este un apel la un alt predicat Prolog. Când programul face acest apel, Prolog testează predicatul apelat să vadă dacă poate fi adevărat. Odată ce subgoal-ul curent a fost satisfăcut (a fost găsit adevărat), se revine și procesul continuă cu următorul subgoal. Dacă procesul a ajuns cu succes la punct, regula a reușit. Pentru a utiliza cu succes o regulă, Prolog trebuie să satisfacă toate subgoal-urile ei, creând o mulțime consistentă de legări de variabile. Dacă un subgoal eșuează (este găsit fals), procesul revine la subgoal-ul anterior și caută alte legări de variabile, și apoi continuă. Acest mecanism se numește **backtracking**.

6. Operatori de egalitate

$X=Y$ verifică dacă X și Y pot fi unificate

- Dacă X este variabilă liberă și Y legată, sau Y este variabilă liberă și X e legată, propoziția este satisfăcută unificând pe X cu Y.
- Dacă X și Y sunt variabile legate, atunci propoziția este satisfăcută dacă relația de egalitate are loc.

$?- [a,b]=[a,b].$	$?- [X,Y]=[a,b].$	$?- [a,b]=[X,Y].$
true	X = a, Y = b.	X = a, Y = b.

$X \neq Y$ verifică dacă X și Y nu pot fi unificate
 $\backslash+ X=Y$

$?- [X,Y,Z] \neq [a,b].$	$?- [X,Y] \neq [a,b].$	$?- [a,b] \neq [X,Y].$
true	false	false
$?- \backslash+ a=a.$	$?- \backslash+ [X,Y]=[a,b].$	$?- \backslash+ [a,b]=[X,Y,Z].$
false	false	true

$X == Y$ verifică dacă X și Y sunt legate la aceeași valoare.

$?- [2,3] == [2,3].$	$?- a == a.$	$?- R == 1.$
true	true	false

$X \neq Y$ verifică dacă X și Y nu au fost legate la aceeași valoare.

$?- [2,3] \neq [3,2].$	$?- a \neq a.$	$?- R \neq 1.$
true	false	true

7. Operatori aritmetici

!!! Important

- $2+4$ e doar o structură, utilizarea sa nu efectuează adunarea
- Utilizarea $2+4$ nu e aceeași ca utilizarea lui 6.

Operatori aritmetici

$=, \backslash, ==, \backslash==$ A se vedea secțiunea 6.

?- $2+4=6$. false	?- $2+4\backslash=6$. true	?- $6==6$. true	?- $6\backslash=7$. true	?- $6==2+4$. false
?- $2+4=2+4$. true	?- $2+4=4+2$. false	?- $X=2+4-1$. $X=2+4-1$.		

$==$

- testează egalitatea aritmetică
- forțază evaluarea aritmetică a ambelor părți
- operanzii trebuie să fie numerici
- variabilele sunt LEGATE

$\backslash=$ testează operatorul aritmetic "diferit"

?- $2+4\backslash=6$. true	?- $2+4\backslash=7$. true	?- $6\backslash=6$. true
--------------------------------	--------------------------------	------------------------------

is

- partea dreaptă este LEGATĂ și numerică
- partea stângă trebuie să fie o variabilă
- dacă variabila este legată, verifică egalitatea numerică (ca și $==$)
- dacă variabila nu este legată, evaluează partea dreaptă și apoi variabila este legată de rezultatul evaluării

?- X is $2+4-1$. $X=5$	?- X is 5. $X=5$
------------------------------	-----------------------

Inegalități

$<$	mai mic
$=<$	mai mic sau egal
$>$	mai mare
$>=$	mai mare sau egal

- evaluează ambele părți
- variabile LEGATE

?- $2+4<5+2$.	?- $2+4\backslash=7$.	?- $6\backslash=6$.
----------------	------------------------	----------------------

true

true

true

Câteva funcții aritmetice predefinite SWI-Prolog

$X \bmod Y$ întoarce restul împărțirii lui X la Y
 $\text{mod}(X, Y)$

$X \text{ div } Y$ întoarce câtul împărțirii lui X la Y
 $\text{div}(X, Y)$

$\text{abs}(X)$ întoarce valoarea absolută a lui X

$\text{sqrt}(X)$ întoarce rădăcina pătrată a lui X

$\text{round}(X)$ întoarce valoarea lui X rotunjită spre cel mai apropiat întreg ($\text{round}(2.56)$ este 3, $\text{round}(2.4)$ este 2)

...

8. Predicate predefinite

$\text{var}(X)$ = adevărat dacă X e liberă, fals dacă e legată

$\text{number}(X)$ = adevărat dacă X e legată la un număr

$\text{integer}(X)$ = adevărat dacă X e legată la un număr întreg

$\text{float}(X)$ = adevărat dacă X e legată la un număr real

$\text{atom}(X)$ = adevărat dacă X e legată la un atom

$\text{atomic}(X)$ = $\text{atom}(X)$ or $\text{number}(X)$

....

9. Predicatul „findall” (determinarea tuturor soluțiilor)

Prolog oferă o modalitate de a găsi toate soluțiile unui predicat în același timp: predicatul **findall**, care colectează într-o listă toate soluțiile găsite.

findall (arg1, arg2, arg3)

Acesta are următoarele argumente:

- primul argument specifică argumentul din predicatul considerat care trebuie colectat în listă;
- al doilea argument specifică predicatul de rezolvat;
- al treilea argument specifică lista în care se vor colecta soluțiile.

10. Negatie - “not”, “\+”

not(subgoal(Arg1, ..., ArgN))

adevărat dacă *subgoal* eșuează(nu se poate demonstra că este adevărat)

\+ subgoal(Arg1, ..., ArgN)

?- \+ (2 = 4).
true.

?- not(2 = 4).
true.

11. Predicatul ! (cut) – “tăietura”

Turbo Prolog conține predicatul cut (!) folosit pentru a preveni backtracking-ul. Când se procesează predicatul !, apelul reușește imediat și se trece la subgoalul următor. O dată ce s-a trecut peste o tăietură, nu este posibilă revenirea la subgoal-urile plasate înaintea ei și nu este posibil backtracking-ul la alte reguli ce definesc predicatul în execuție.

Există două utilizări importante ale tăieturii:

1. Când știm dinainte că anumite posibilități nu vor duce la soluții, este o pierdere de timp să lăsăm sistemul să lucreze. În acest caz, tăietura se numește **tăietură verde**.
2. Când logica programului cere o tăietură, pentru prevenirea luării în considerație a subgoal-urilor alternative, pentru a evita obținerea de soluții eronate. În acest caz, tăietura se numește **tăietură roșie**.

Prevenirea backtracking-ului la un subgoal anterior

În acest caz tăietura se utilizează astfel: `r1 :- a, b, !, c.`

Aceasta este o modalitate de a spune că suntem mulțumiți cu primele soluții descoperite cu subgoal-urile a și b. Deși Prolog ar putea găsi mai multe soluții prin apelul la c, nu este autorizat să revină la a și b. De-asemenea, nu este autorizat să revină la altă clauză care definește predicatul r1.

Prevenirea backtracking-ului la următoarea clauză

Tăietura poate fi utilizată pentru a-i spune sistemului Prolog că a ales corect clauza pentru un predicat particular. De exemplu, fie codul următor:

```
r(1) :- !, a, b, c.  
r(2) :- !, d.  
r(3) :- !, e.  
r(_) :- write("Aici intră resul apelurilor").
```

Folosirea tăieturii face predicatul r determinist. Aici, Prolog apelează predicatul r cu un argument întreg. Să presupunem că apelul este r(1). Turbo Prolog caută o potrivire a apelului. O găsește la prima clauză. Faptul că imediat după intrarea în clauză urmează o tăietură, împiedică Prolog să mai caute și alte potriviri ale apelului r(1) cu alte clauze.

Observație. Acest tip de structură este echivalent cu o instrucțiune de tip *case* unde condiția de test a fost inclusă în capul clauzei. La fel de bine s-ar fi putut spune și

```
r(X) :- X = 1, !, a, b, c.  
r(X) :- X = 2, !, d.  
r(X) :- X = 3, !, e.  
r(_) :- write("Aici intra restul apelurilor").
```

Notă. Deci, următoarele secvențe sunt echivalente:

case X of	
v1: corp1;	predicat(X) :- X = v1, !, corp1.
v2: corp2;	predicat(X) :- X = v2, !, corp2.
...	...
else corp_else.	predicat(X) :- corp_else.

De asemenea, următoarele secvențe sunt echivalente:

if cond1 then	predicat(...) :-
corp1	cond1, !, corp1.
else if cond2 then	predicat(...) :-
corp2	cond2, !, corp2.
...	...
else	predicat(...) :-
corp_else.	corp_else.

12. Predicatul “fail”

Valoarea lui *fail* este eșec. Prin aceasta el încurajează backtracking-ul. Efectul lui este același cu al unui predicat imposibil, de genul $2 = 3$. Fie următorul exemplu:

```
predicat(a, b).
predicat(c, d).
predicat(e, f).
toate :-
    predicat(X, Y),
    write(X),write(Y),nl,
    fail.
toate1 :-
    predicat(X, Y),
    write(X),write(Y),nl.
```

Predicatele **toate** și **toate1** sunt fără parametri, și ca atare sistemul va trebui să răspundă dacă există X și Y astfel încât aceste predicate să aibă loc.

?- toate.	?-toate1.
ab	ab
cd	true ;
ef	cd
false.	true ;
	ef
	true.

?-predicat(X,Y).
 X = a,
 Y = b ;
 X = c,
 Y = d ;
 X = e,
 Y = f.

Faptul că apelul predicatului **toate** se termină cu *fail* (care eșuează întotdeauna) obligă Prolog să înceapă backtracking prin corpul regulii **toate**. Prolog va reveni până la ultimul apel care poate oferi mai multe soluții. Predicatul **write** nu poate da alte soluții, deci revine la apelul lui **predicat**.

Observații.

- Acel **false** de la sfârșitul soluțiilor semnifică faptul că predicatul ‘toate’ nu a fost satisfăcut.
- După *fail* nu are rost să puneți nici un predicat, deoarece Prolog nu va ajunge să-l execute niciodată.

Notă. Secvențele următoare sunt echivalente:

cât timp *condiție* execută
corp

predicat :-
condiție,
corp,
fail.

13. Obiecte simple și obiecte compuse

Obiecte simple

Un obiect simplu este fie o variabilă, fie o constantă. O constantă este fie un caracter, fie un număr, fie un atom (simbol sau string).

Variabilele Prolog sunt locale, nu globale. Adică, dacă două clauze conțin fiecare câte o variabilă numită X, cele două variabile sunt distincte și, de obicei, nu au efect una asupra celeilalte.

Obiecte compuse și functori

Obiectele compuse ne permit să tratăm mai multe informații ca pe un singur element, într-un astfel de mod încât să-l putem utiliza și pe bucăți. Fie, de exemplu, data de *2 februarie 1998*. Constă din trei informații, ziua, luna și anul, dar e util să o tratăm ca un singur obiect cu o structură arborescentă:

DATA
 / | \
 2 februarie 1998

Acest lucru se poate face scriind obiectul compus astfel:

`data(2, "februarie", 1998)`

Aceasta seamănă cu un fapt Prolog, dar nu este decât un obiect (o dată) pe care îl putem manevra la fel ca pe un simbol sau număr. Din punct de vedere sintactic, începe cu un nume (sau **functor**, în acest caz cuvântul **data**) urmat de trei argumente.

Notă. Functorul în Prolog nu este același lucru cu funcția din alte limbaje de programare. Este doar un nume care identifică un tip de date compuse și care ține argumentele laolaltă.

Argumentele unei date compuse pot fi chiar ele compuse. Iată un exemplu:
`naștere(persoana("Ioan", "Popescu"), data(2, "februarie", 1918))`

Unificarea obiectelor compuse

Un obiect compus se poate unifica fie cu o variabilă simplă, fie cu un obiect compus care se potrivește cu el. De exemplu,

`data(2, "februarie", 1998)`

se potrivește cu variabila liberă *X* și are ca rezultat legarea lui *X* de `data(...)`. De asemenea, obiectul compus de mai sus se potrivește și cu

`data(Zi, Lu, An)`

și are ca rezultat legarea variabilei *Zi* de valoarea 2, a variabilei *Lu* de valoarea "februarie" și a variabilei *An* de valoarea 1998.

Observații

- Convenim să folosim următoarea declarație pentru *specificarea* unui domeniu cu alternative
 `% domeniu = alternativa1(dom, dom, ..., dom);`
 `%           alternativa2(dom, dom, ..., dom);`
 `%           ...`
- Functorii pot fi folosiți pentru controla argumentele care pot avea tipuri multiple
 `% element = i(integer); r(real); s(string)`

14. Optimizarea prin recursivitate de coadă (tail recursion)

Recursivitatea are o mare problemă: consumă multă memorie. Dacă o procedură se repetă de 100 ori, 100 de stadii diferite ale execuției procedurii (cadre de stivă) sunt memorate.

Totuși, există un caz special când o procedură se apelează pe ea fără să genereze cadru de stivă. Dacă procedura apelatoare apelează o procedură ca ultim pas al sau (după acest apel urmează punctul). Când procedura apelată se termină, procedura apelatoare nu mai are altceva de făcut. Aceasta înseamnă că procedura apelatoare nu are sens să-și memoreze stadiul execuției, deoarece nu mai are nevoie de acesta.

Funcționarea recursivității de coadă

Iată două reguli depre cum să faceți o recursivitate de coadă:

1. Apelul recursiv este ultimul subgoal din clauza respectivă.
2. Nu există puncte de backtracking mai sus în acea clauză (adică, subgoal-urile de mai sus sunt deterministe).

Iată un exemplu:

```
tip(N) :-  
    write(N),  
    nl,  
    Nou is N + 1,  
    tip(Nou).
```

Această procedură folosește recursivitatea de coadă. Nu consumă memorie, și nu se oprește niciodată. Eventual, din cauza rotunjirilor, de la un moment va da rezultate incorecte, dar nu se va opri.

Exemple greșite de recursivitate de coadă

Iată cateva reguli despre cum să NU faceți o recursivitate de coadă:

1. Dacă apelul recursiv nu este ultimul pas, procedura nu folosește recursivitatea de coadă.

Exemplu:

```
tip (N) :-  
    write(N),  
    nl,  
    Nou is N + 1,  
    tip (Nou),  
    nl.
```

2. Un alt mod de a pierde recursivitatea de coadă este de a lăsa o alternativă neîncercată la momentul apelului recursiv.

Exemplu:

```
tip(N) :-  
    write(N),  
    nl,  
    Nou is N + 1,  
    tip(Nou).  
tip(N) :-  
    N < 0,  
    write('N este negativ.').
```

Aici, prima clauză se apelează înainte ca a doua să fie încercată. După un anumit număr de pași intră în criză de memorie.

3. Alternativa neîncercată nu trebuie neapărat să fie o clauză separată a procedurii recursive. Poate să fie o alternativă a unei clauze apelate din interiorul procedurii recursive.

Exemplu:

```
tip (N) :-  
    write(N),  
    nl,  
    Nou is N + 1,  
    verif(Nou),  
    tip(Nou).  
verif(Z) :- Z >= 0.  
verif(Z) :- Z < 0.
```

Dacă N este pozitiv, prima clauză a predicatului **verif** a reușit, dar a doua nu a fost încercată. Deci, **tip** trebuie să-și pastreze o copie a cadrului de stivă.

Utilizarea tăieturii pentru păstrarea recursivității de coadă

A doua și a treia situație de mai sus pot fi înlăturate dacă se utilizează tăietura, chiar dacă există alternative neîncercate.

Exemplu la situația a doua:

```
tip (N) :-  
    N >= 0,  
    !,  
    write(N),  
    nl,  
    Nou = N + 1,  
    tip(Nou).  
tip(N) :-  
    N < 0,  
    write("N este negativ.").
```

Exemplu la situația a treia:

```
tip(N) :-  
    write(N),  
    nl,  
    Nou = N + 1,  
    verif(Nou),  
    !,  
    tip(Nou).  
verif(Z) :- Z >= 0.  
verif(Z) :- Z < 0.
```